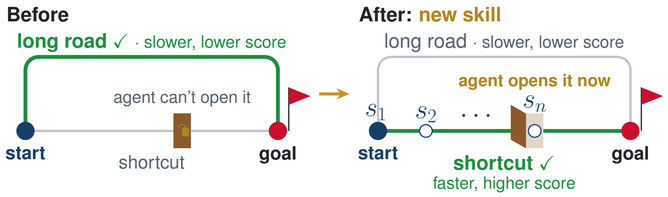
Australian National University

One Solve, Every Step:
Accelerating Actor–Critic Learning

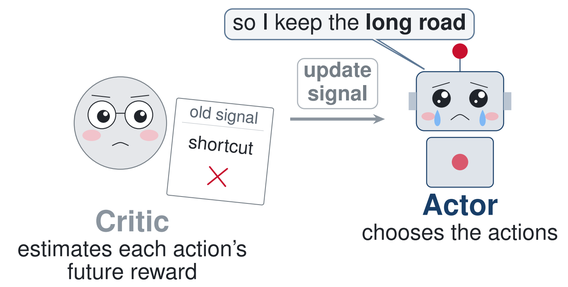
Hanning Deng
Supervisor: Prof. Liang Zheng
School of Computing, ANU
COMP8800 · S2 2026

01 Problem: new skill, old choices

An agent goes from start to goal. Arriving sooner scores higher.



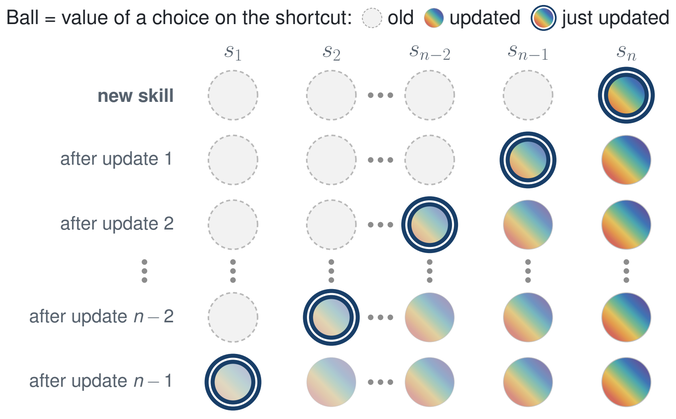
Now the agent has learned to open the door (**new skill**), so the shortcut is faster. Why does it still take the long road?



The update signal tells the actor which actions to choose more or less often. **Problem:** the skill was learned at the door; back at the start, the signal still **undervalues the shortcut**. Real agents keep learning new skills; if the actor needs many updates to catch up each time, training is slow and costly.

02 Why: the critic learns step by step

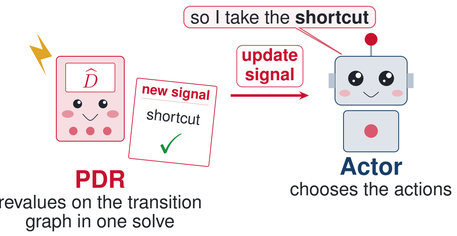
The critic is trained by **temporal-difference (TD) learning**, the usual way: each step learns its value from the next step. So a new value reaches earlier steps only gradually.

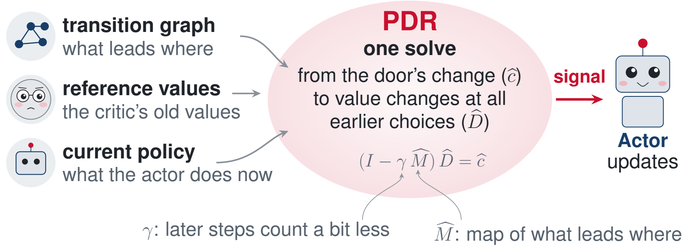


In this update order, the new value moves back one step per critic update: a choice d steps back waits ***d* updates**.

03 Our solution: PDR revalues in one solve

We propose **Policy Differential Revaluation (PDR)**. It reuses the agent's past experience:
Past steps: after each action, where the agent went and what reward it got.
Transition graph: these steps linked into a map of what leads where, as in **01**.
PDR: the actor now opens the door, and the critic's old values already show what that gains there: the door's change ($\hat{c}$). Instead of passing this news back one step at a time, PDR works out the value change of **every earlier choice together**, like solving a set of equations at once: **one solve**.





What the actor learns from

Q_{ref} (critic's old values) + $\hat{D}$ (PDR's change) = **up-to-date values** (at **every step**, from one solve)



In **02**, a choice d steps back waits d updates. Here, one solve gives every choice a **re-estimated value**.

Mechanism evidence: why TD is slow

Test: a 50-step chain where one step's value changes. We count the work (values read) until a choice d steps back is 90% updated.



In this update order, TD requires more work for choices farther from the change. To update a choice 45 steps away, TD needs **53×** more work than PDR. PDR's work is the same at every distance.

Cost: after optimisation, PDR trains in **1.04–1.87×** TD's time.

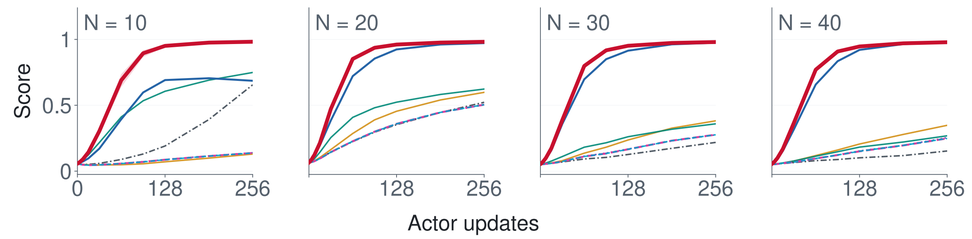
04 Results: PDR adapts fastest on public benchmarks

The actor should learn faster when its update signal is not only correct, but also timely. All methods start from the same saved agent. **On DeepSea, PDR reaches a score of 0.7 (max 1) with 16–128× fewer updates than TD.**

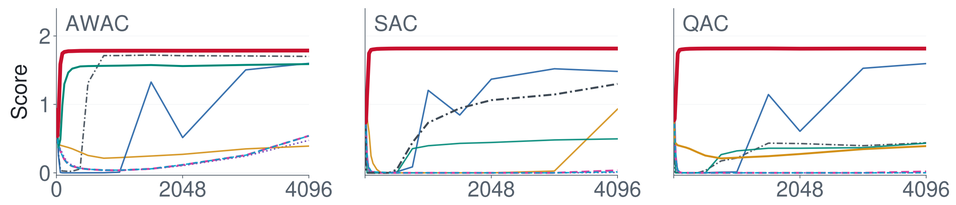
TD · DisCor · IQL · PS (prioritized sweeping)
PER · 8× TD · CQL · **PDR**

Lines: mean of many runs; red band: PDR's 95% confidence interval. *Score*: reward per try. *Success*: share of tries that finish. *Actor updates*: times the actor is adjusted. 8× TD: 8× more critic updates. PS also uses the graph.

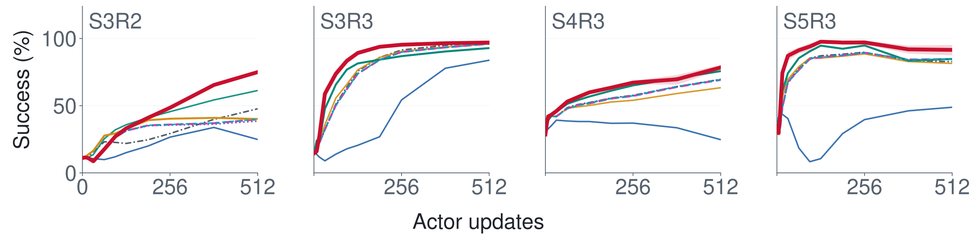
DeepSea: reach the treasure in the far corner; it takes N correct moves in a row. Start: a pretrained agent that reaches it in only 5–15% of tries. **PDR reaches high scores earlier at every size, up to N = 40.**



MiniGridLoCA, a reward change: the old goal paid 4, now 1; the new goal pays 2. AWAC, SAC, QAC: three actor–critic backbones. **PDR generalises: it reaches high scores earlier and ends highest on all three.**



BabyAI KeyCorridor: find the key, unlock the door, pick up the ball; four task variants. Start: a saved, partly trained agent. **PDR reaches high scores earlier on three, and has the highest success over training on all four.**



Real rollouts, TD vs PDR (▶ = agent): same start, same number of updates, frame at the same move.



PDR helps the actor adapt sooner by revaluing all earlier choices together.